OnDemand

HIGH-LEVEL ARCHITECTURE

# OnDemand Platform Architecture

How the agentic AI platform is structured, how its major components interact, and how it is delivered and operated.

CLASSIFICATION

Confidential

AUDIENCE

Authorized stakeholders

CONTENTS


01 About this document
02 Executive summary
03 Platform capabilities
04 Architecture principles
05 System context
06 Logical architecture
07 Core capability architecture
08 Integration and communication
09 Data architecture
10 Identity, access and tenancy
11 Security architecture
12 Infrastructure and deployment
13 Delivery and release management
14 Observability, resilience and continuity
15 Architecture roadmap
16 Appendix

01

# About this document

This document describes the high-level architecture of the OnDemand platform: what it does, how it is organised, how its major parts work together, and how it is built, deployed and operated.

It is written for customers, partners and other stakeholders who need to understand how the platform is structured, for example during evaluation, integration planning or security and compliance review. It stays at the level of architecture and leaves out implementation detail.

| Item | Detail |
| --- | --- |
| Scope | Logical architecture, core capability flows, data and integration patterns, identity and security model, infrastructure, delivery and operations. |
| Audience | Customers, partners and other stakeholders, including their architects, technical teams and security and compliance reviewers. |
| Classification | Confidential For authorized stakeholders. Do not redistribute without approval. |

02

# Executive summary

OnDemand is a multi-tenant agentic AI platform. Organisations use it to chat with AI agents, connect those agents to their own systems and data, automate work with visual workflows, and deploy their own models and applications, all behind one set of APIs.

A user's request is not sent straight to a single model. The platform **plans** it, breaks it into steps, runs each step with the right **agent** (an API, a knowledge base, a device, an MCP server or the autonomous OnDemand Agent) and then writes the answer with the model the organisation has chosen. Results stream back to the user as they are produced.

The platform is built as a set of independent services grouped by business domain. The services talk to each other through typed contracts and an event bus, run on Kubernetes in Microsoft Azure, and are released through an automated, security-gated GitOps pipeline. The same product can run as a managed cloud service or inside a customer's own environment, including air-gapped sites.

### Agentic by design

Requests are planned and executed step by step across tools, with human clarification when needed.

### Model-agnostic

Leading commercial models and customer-hosted models are reached through one endpoint abstraction.

### Multi-tenant

Every request, record and knowledge base is scoped to an organisation, with role-based access inside it.

### Streaming and resumable

Long-running AI work streams live and survives reloads, disconnects and session switches.

### Event-driven

Usage, billing, analytics and notifications react to domain events instead of tight coupling.

### Deployable anywhere

Managed cloud, customer cloud or on-premises, online or air-gapped.

03

# Platform capabilities

The platform offers nine capability areas through a web console, public APIs and embedded channels. Administration covers organisations, users and roles, API keys, usage limits, billing and invoices.

### Playground & conversational AI

Chat with agents and models, attach files and knowledge, see the plan and each step as it runs, and continue work across sessions and projects.

### Agents & marketplace

Turn REST APIs, knowledge bases, IoT devices, MCP servers and data connectors into agents. Publish them to a marketplace where authors earn rewards.

### Skills

Reusable playbooks that teach agents how an organisation wants work done. They are security-scanned before use and can be shared.

### OnDemand Agent

The platform's autonomous agent. In an isolated sandbox it writes and runs code, browses the web, uses platform tools and delivers files and reports.

### Workflow automation

Visual flows started by API, schedule, webhook or email, with AI steps, human approval gates and delivery to email, Slack, webhooks or phone.

### Voice agents

Inbound and outbound phone agents using real-time speech models, usable on their own or inside workflows.

### Knowledge & media

Ingest documents, audio, video and connected drives; extract, transcribe and embed them into per-tenant knowledge for retrieval.

### Serverless & bring-your-own

Deploy containerised applications from Git, host MCP servers, deploy open models on GPUs, or register your own inference endpoints.

### Integration surface

Public APIs and SDKs with scoped keys, an embeddable web chatbot, Slack and email channels, and batch processing.

04

# Architecture principles

These principles explain most of the design decisions described later in this document.

| Principle | What it means in practice |
| --- | --- |
| Domain-aligned services | Each business domain (conversations, agents, knowledge, billing, identity, automation) is owned by its own service with its own data store. Other services reach that data only through the owning service's interface. |
| Contracts first | Service-to-service calls use versioned, typed RPC contracts kept in a shared repository and generated for each language in use. |
| Asynchronous by default for heavy work | Anything slow or bursty (AI fulfilment, media processing, webhooks, billing) runs on durable queues and event streams, never inside a user's HTTP request. |
| Stream everything the user waits for | Progress, reasoning and answers are pushed live. Every event is indexed so clients can reconnect without losing output. |
| Endpoints, not vendors | Models are addressed by platform endpoint IDs, so providers and customer-hosted models can be swapped without changing callers. |
| Tenant isolation everywhere | The organisation is part of every request, record, key and vector namespace. Plan and rate-limit checks happen before any billable work. |
| Git is the source of truth | Application releases and gateway configuration are declared in Git and pulled into clusters. Nothing pushes into production by hand. |
| Portable by construction | Storage, secrets and compute sit behind adapters, so the same build runs in the managed cloud, in customer clouds and on-premises. |

05

# System context

OnDemand sits between the people and systems that use AI and the providers that supply models, infrastructure and business services. Users reach it through the web console, APIs and embedded channels. The platform in turn relies on model providers, cloud infrastructure and a small set of specialist services for payments, communications, data access and observability.

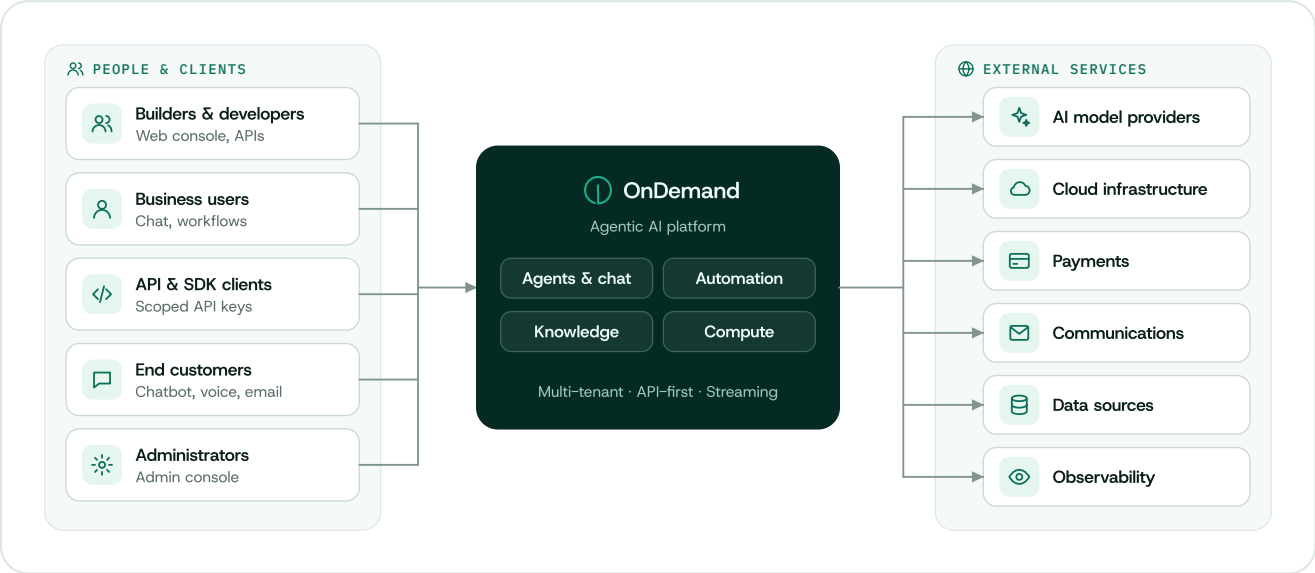



FIGURE 1 System context: who uses the platform and which external services it depends on.

| External dependency | Role |
|---|---|
| AI model providers | Commercial chat, reasoning, vision, speech, image and embedding models from several major providers, plus customer-hosted open models. |
| Cloud infrastructure | Kubernetes, container registry, managed cache, object storage, key management, serverless containers and GPU capacity (Microsoft Azure first). |
| Payments | Subscriptions, cards, invoices and prepaid credit through a payments provider. |
| Communications | Transactional email, push notifications, telephony for voice agents and Slack. |
| Data sources | Customer drives, document libraries, object stores, code repositories, web search and transcription services. |
| Observability | Metrics, logs, traces, APM and product analytics. |

06

# Logical architecture

The platform is organised in layers. Clients reach one API gateway. Behind it, six groups of domain services do the work. The services communicate through typed contracts, an event bus, job queues and live streams, and each keeps its own data.

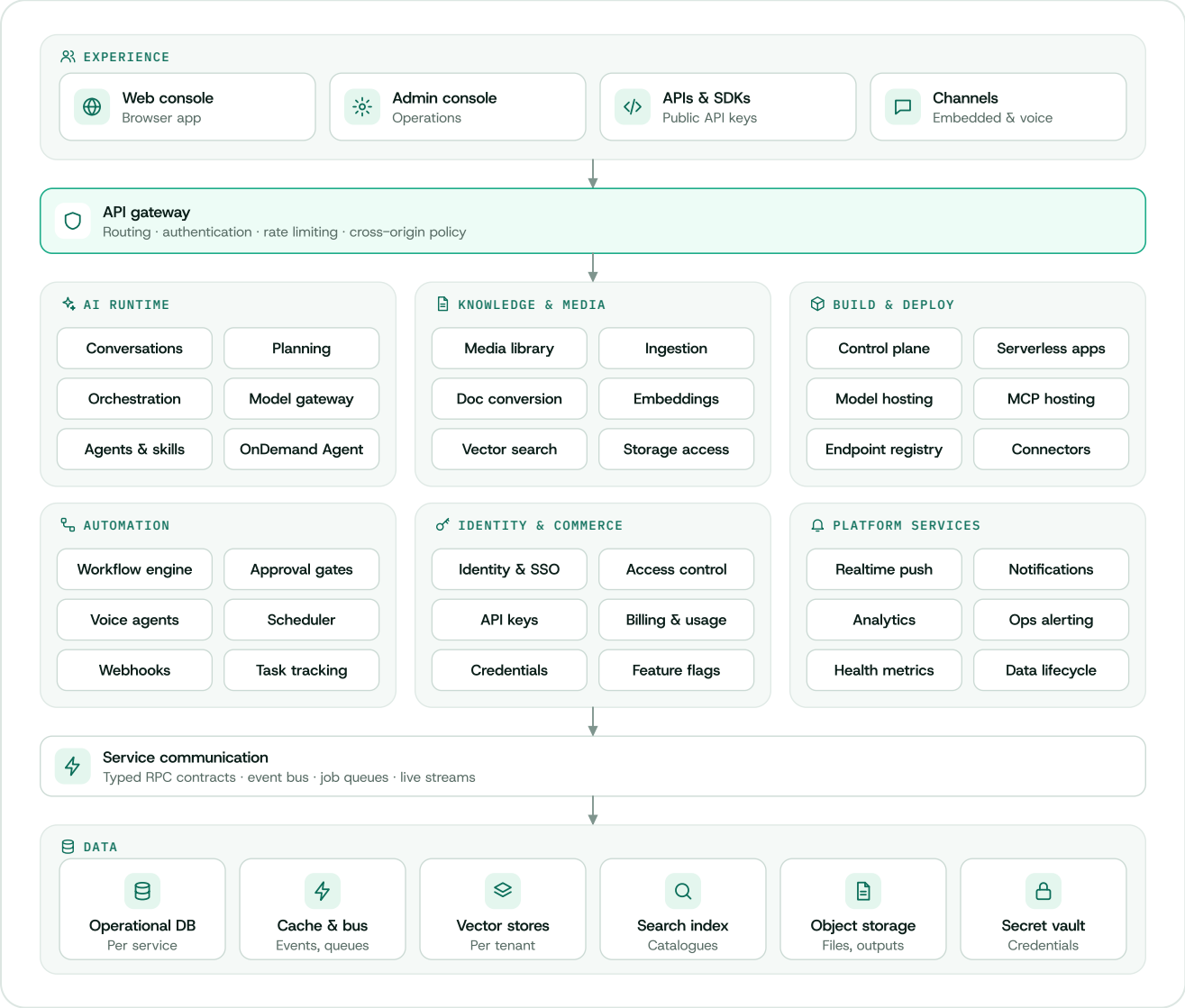



FIGURE 2 Logical architecture: experience layer, edge, domain services, communication fabric and data.

| Domain | Responsibility |
|---|---|
| AI runtime | Conversations and sessions; query planning and step orchestration; the model gateway; the agent and skill runtime; the OnDemand Agent sandbox. |
| Knowledge & media | Media library and batches; ingestion, conversion, OCR and transcription; embeddings and vector search; storage access through signed URLs. |
| Build & deploy | The control plane for model endpoints and reasoning modes, serverless applications, hosted models (BYOM), hosted MCP servers and data connectors. |

| Domain | Responsibility |
| --- | --- |
| **Automation** | The workflow engine, approval gates, voice agents, scheduling, outbound webhooks and background task tracking. |
| **Identity & commerce** | Sign-in and SSO, organisations, roles and permissions, API keys, credential vault, plans, credits, usage metering and billing. |
| **Platform services** | Realtime push to users, email and push notifications, usage analytics, operational alerting, health metrics and data lifecycle clean-up. |

07

# Core capability architecture

This section follows the six most important capabilities through the architecture.

## Conversational query processing

A query from the Playground, an API caller, a workflow or a channel goes through the same pipeline. The conversation service accepts it, checks the organisation's plan and rate limits, gathers context (history, attached agents, skills, knowledge and model settings) and puts a job on a durable queue. A worker then runs the query through the **orchestration engine**.

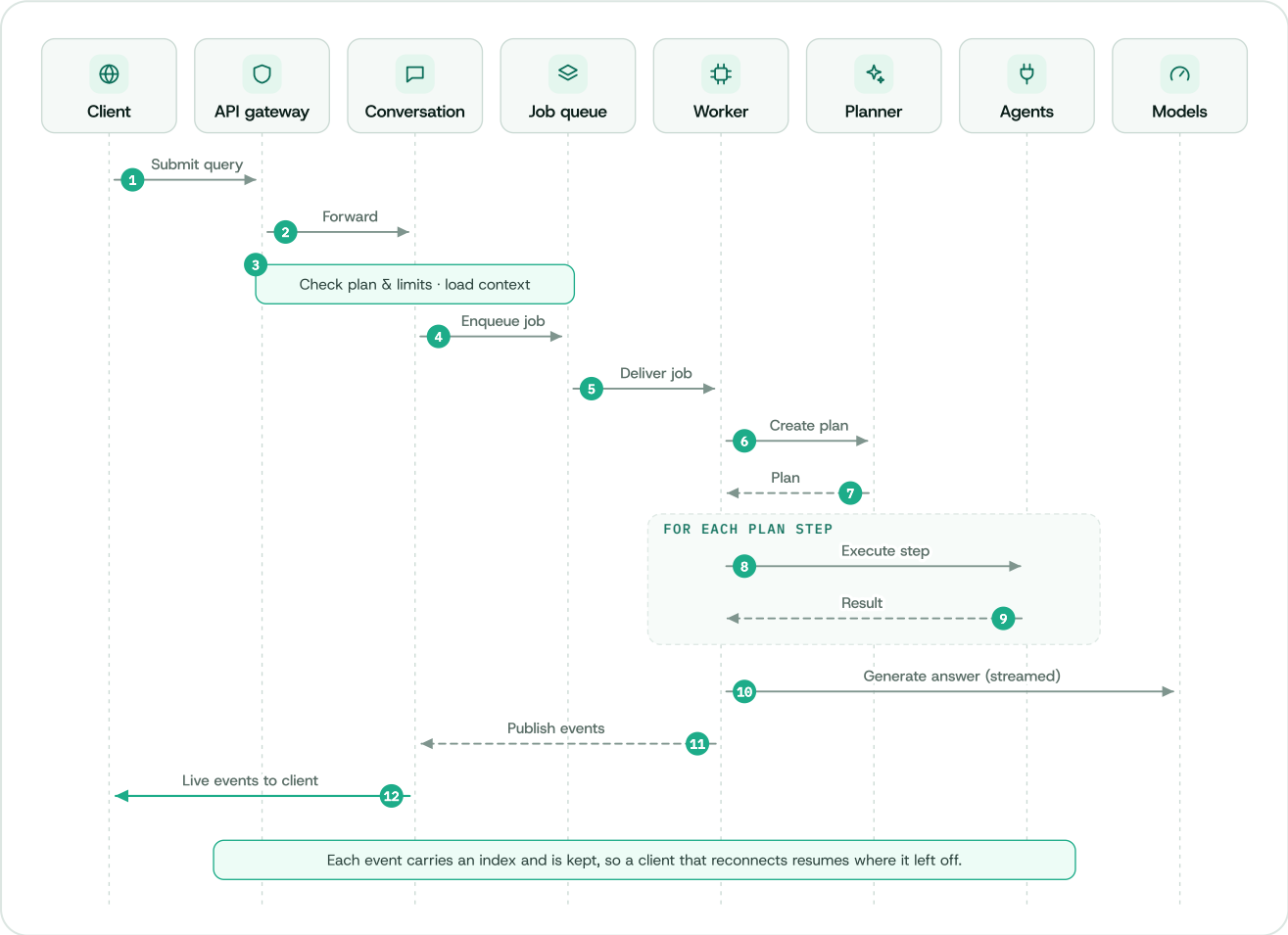



FIGURE 3 Conversational query: planning, step execution and streamed fulfilment with resumable delivery.

- **Plan.** A planner picks the agents and steps needed. If the request is ambiguous, the run pauses and asks the user a clarifying question, then continues with the answer.
- **Execute.** Steps run in dependency order, and independent steps can run in parallel. Each step calls an agent through the agent runtime.
- **Fulfil.** The chosen model writes the final answer from the step results and streams it token by token.

- **Deliver.** Results arrive as a live stream, a synchronous response, a webhook callback or an asynchronous job, depending on how the query was submitted.

A run is always in one of these states: `planning` `executing` `paused` `fulfilling` `completed` `failed` `cancelled`. Users can cancel, regenerate or fork a conversation at any time. **Live sessions** let an agent keep working on a topic in the background and report back by email. **Batch** processing runs large sets of queries asynchronously.

RESILIENCE

Every streamed event is numbered and kept. If a browser reloads, loses its connection or switches sessions, it asks for events after the last number it received, so no output is lost or duplicated.

## Agents, skills and MCP

In OnDemand an **agent** is a tool the planner can use. Organisations build agents from their own systems or subscribe to agents in the marketplace. The agent runtime handles the registry, credentials, schema-aware calling and usage metering, so every agent type behaves the same way to the planner.

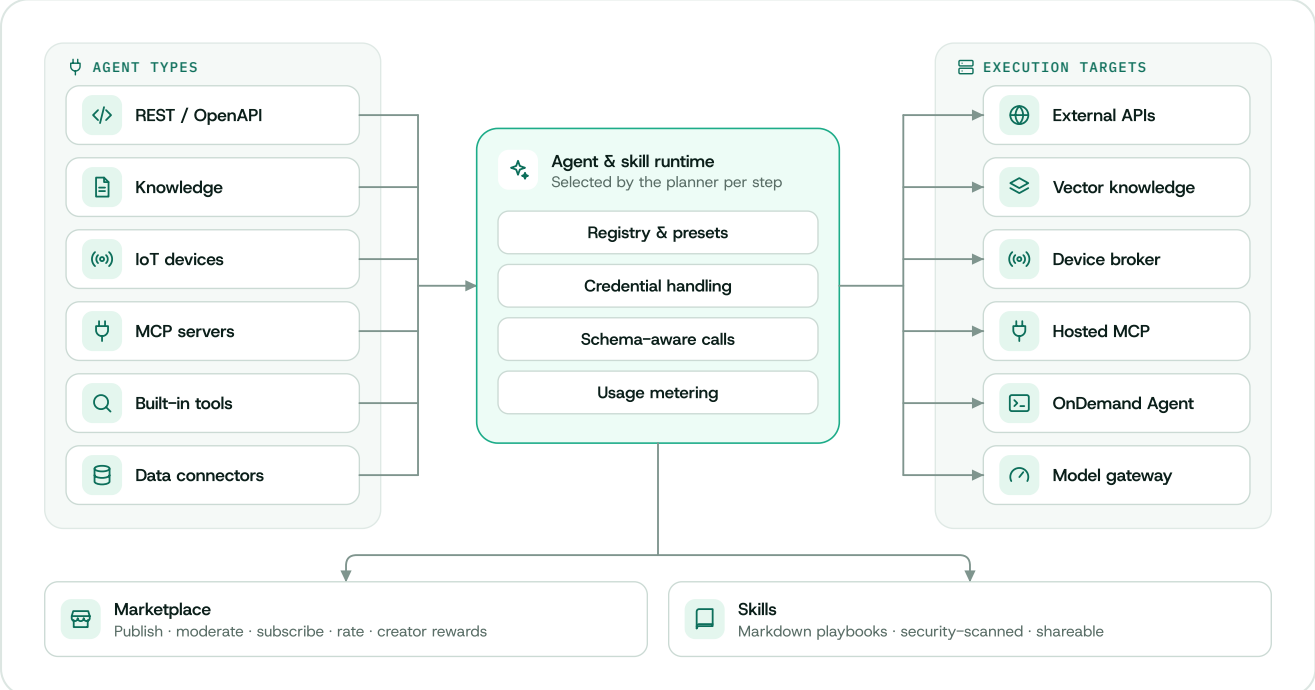



FIGURE 4 Agents and skills: agent types, the shared runtime and where calls are executed.

- **REST / OpenAPI agents** are imported from an API specification. The platform tests and repairs the specification, supports no-auth, API-key and OAuth access, and guards outbound calls against server-side request forgery.
- **Knowledge agents** answer from an organisation's own documents through vector search.

- **IoT agents** read from and act on devices through a managed MQTT broker.
- **MCP agents** connect to third-party Model Context Protocol servers or to servers hosted on OnDemand. Hosted servers are built from a Git repository or a template and run as isolated workloads that shut down when idle.
- **Skills** are written playbooks, optionally with helper files, that agents follow. Every skill is scanned before it can be used. Skills start private to the organisation and can be published.
- **Marketplace** listings are moderated before they go public. Usage of a published agent earns its author rewards in a creator wallet.

## Workflow automation

The Agents Flow Builder is a visual editor for automations. A workflow is a graph. A trigger starts a run, input and AI steps produce results, an analyzer checks them, an optional approval gate waits for a person, and delivery nodes send the outcome where it is needed. AI steps use the same query pipeline as the Playground, so every agent and skill is available inside workflows.

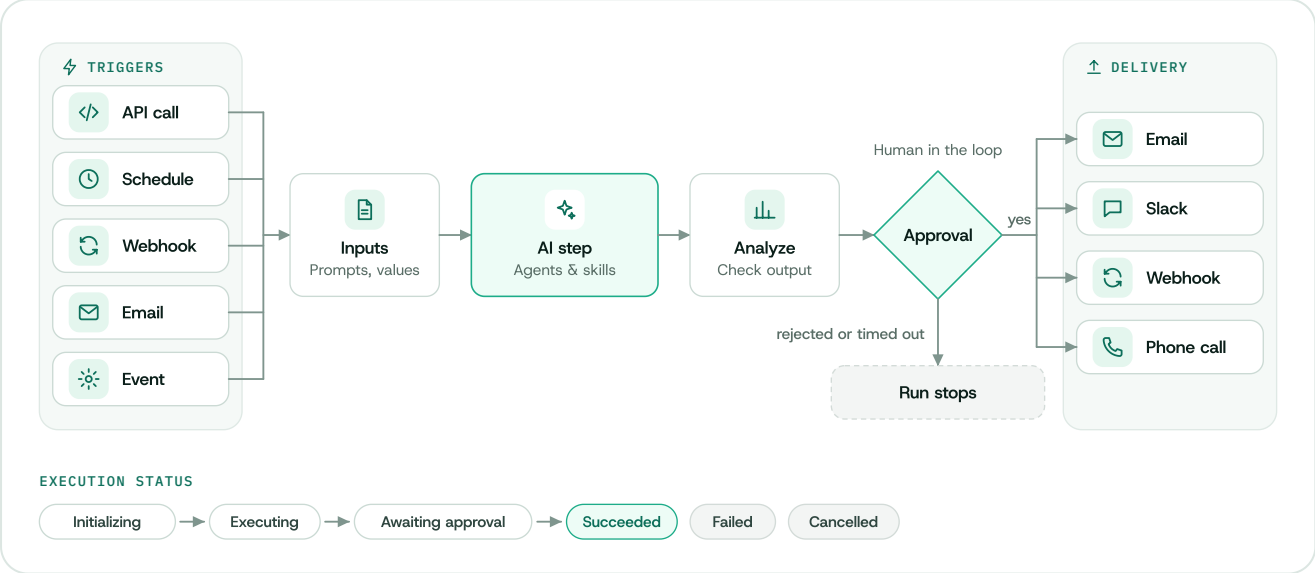



FIGURE 5 Workflow automation: triggers, AI steps, human approval and delivery, with the execution lifecycle.

- **Triggers:** authenticated API calls, schedules (simple or advanced), webhooks, incoming email on connected mailboxes, and platform events.
- **Human in the loop:** approval gates pause a run until an authorised person approves or rejects it, with a timeout.
- **Delivery:** email with delivery tracking, Slack, webhooks and outbound phone calls. Voice steps use real-time speech models over telephony.
- **Operations:** every run keeps a full log and output history. Workflows can be shared as templates in a workflow marketplace.

## OnDemand Agent (autonomous agent)

For open-ended tasks (analysing data, producing reports, writing code, researching on the web) the planner can hand a step to the **OnDemand Agent**. It runs in an isolated, per-task sandbox with a shell, a file system, a web browser and a library of skills. It can also call back into the platform's own agents and data through a tenant-scoped MCP interface.

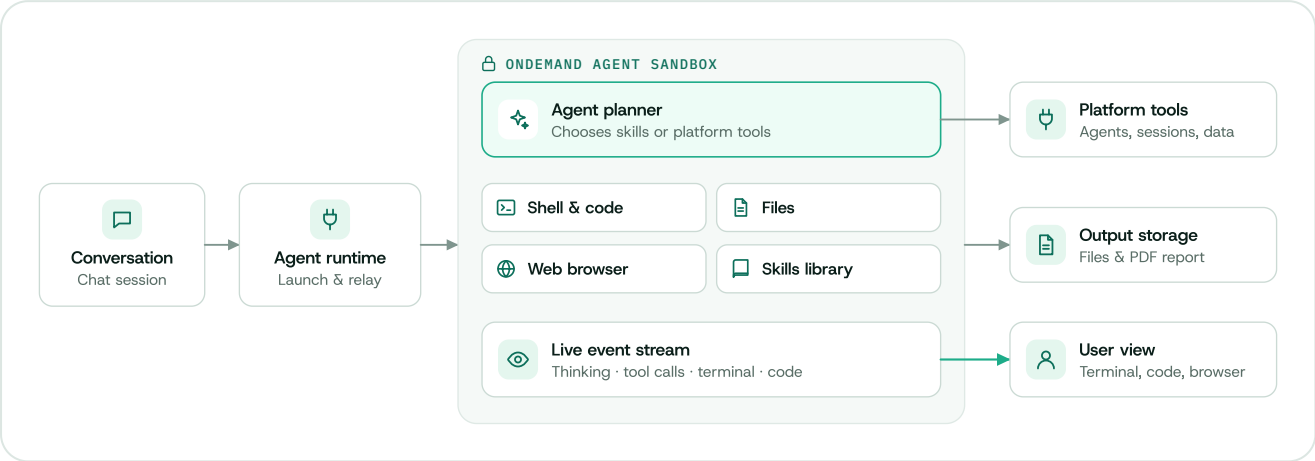



FIGURE 6 OnDemand Agent: sandboxed autonomous execution with live visibility and stored outputs.

Users watch the agent work in real time: its reasoning, tool calls, terminal output, generated code and a live browser view. When a tool needs credentials the user has not supplied, the agent asks for them in the interface. Generated files and a formatted report are saved to object storage. Executions can be shared or resumed.

## Knowledge and media

Files reach the platform through direct upload, connected drives and document libraries, or web and video links. Uploads go straight to object storage through short-lived signed URLs. The media service records each item and hands processing to background workers.

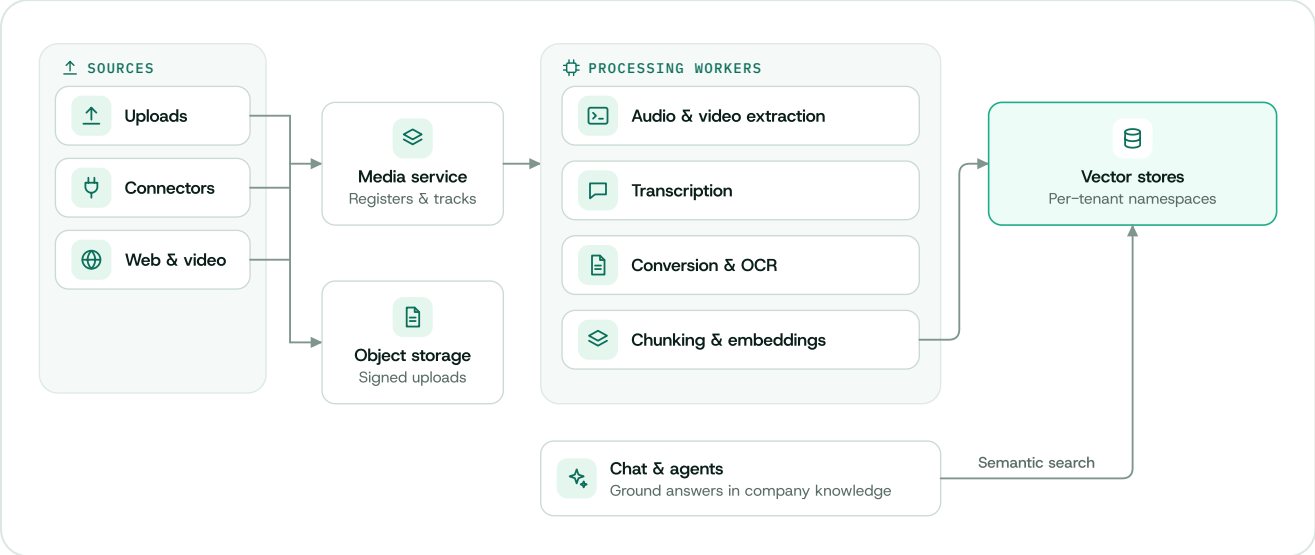



FIGURE 7 Knowledge and media: ingestion, processing and per-tenant semantic retrieval.

Workers extract audio, transcribe speech, convert and OCR documents, then split the content into chunks and embed them into vector stores. Each organisation has its own namespaces. Progress appears in the user's task tray, and completion can trigger a customer webhook. Conversations and knowledge agents then use semantic search to ground answers in the organisation's own content.

## Serverless applications and bring-your-own compute

Organisations can run their own code and models next to the platform's AI services.

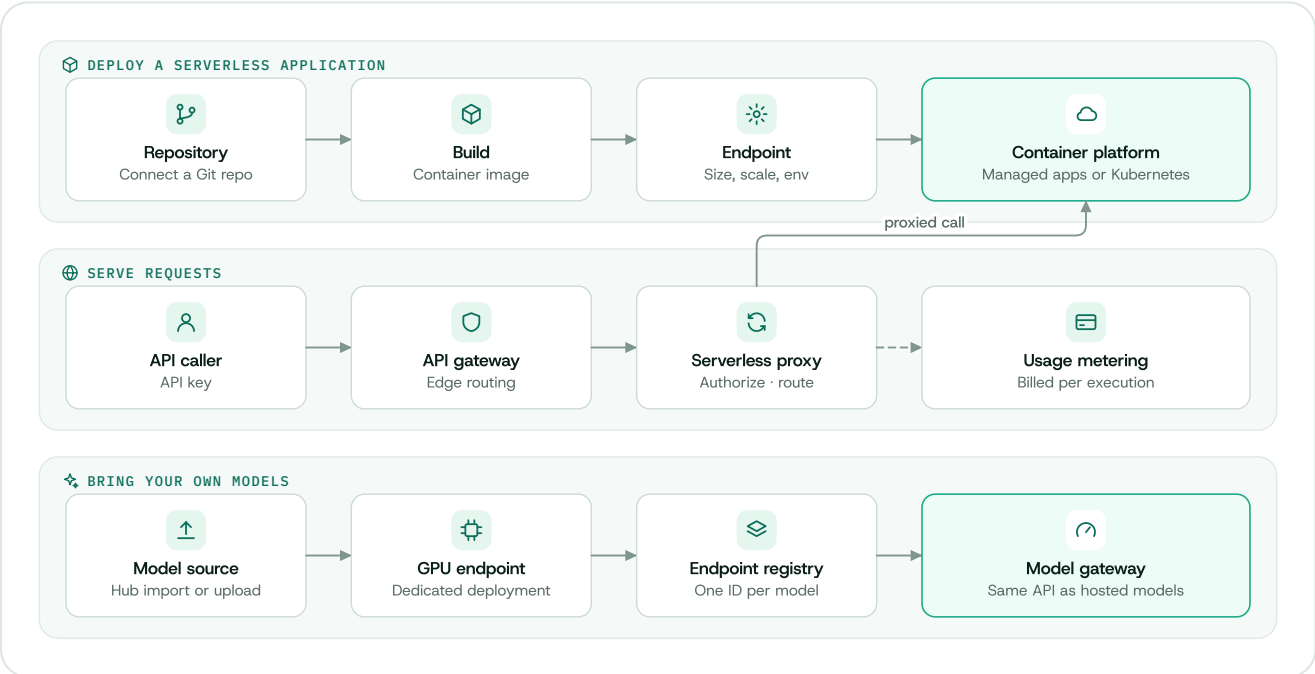



FIGURE 8 Serverless and bring-your-own compute: deploy from Git, serve through the platform, and register customer models.

- **Serverless applications:** connect a repository, build a container image, then create an endpoint with the size, scale and settings it needs. The platform runs it on managed container apps in the cloud, or on Kubernetes on-premises, and provides history, logs, restart and rollback.
- **Serving:** callers reach an endpoint through the API gateway and a serverless proxy that authorises the call, routes it and records usage for billing.
- **Bring your own model (BYOM):** open models imported from a model hub or uploaded, then deployed on dedicated GPU capacity.
- **Bring your own inference (BYOI):** an existing inference API registered as an endpoint.
- **Custom reasoning modes:** an organisation can tune how the planner reasons. Every model, whether hosted or customer-owned, is reached through the same model gateway.

08

# Integration and communication

Services use four communication patterns. Each one is chosen for the job it does.

| Pattern | Used for | Characteristics |
|---|---|---|
| **Typed RPC** | Direct questions between services: validate access, resolve a model endpoint, run an agent | Synchronous, versioned contracts, deadline-bound, traced |
| **Event bus** | Facts other domains react to: usage, media lifecycle, account changes, schedules | Durable topics with consumer groups; at-least-once delivery; producers do not know their consumers |
| **Job queues** | Heavy or retryable work: AI fulfilment, orchestration steps, exports, webhooks, clean-ups | Durable, retried with back-off, scheduled, observable |
| **Live streams** | Pushing progress to users: query events, agent activity, notifications, log tails | Server-sent events to clients; publish/subscribe between workers and front-end services |

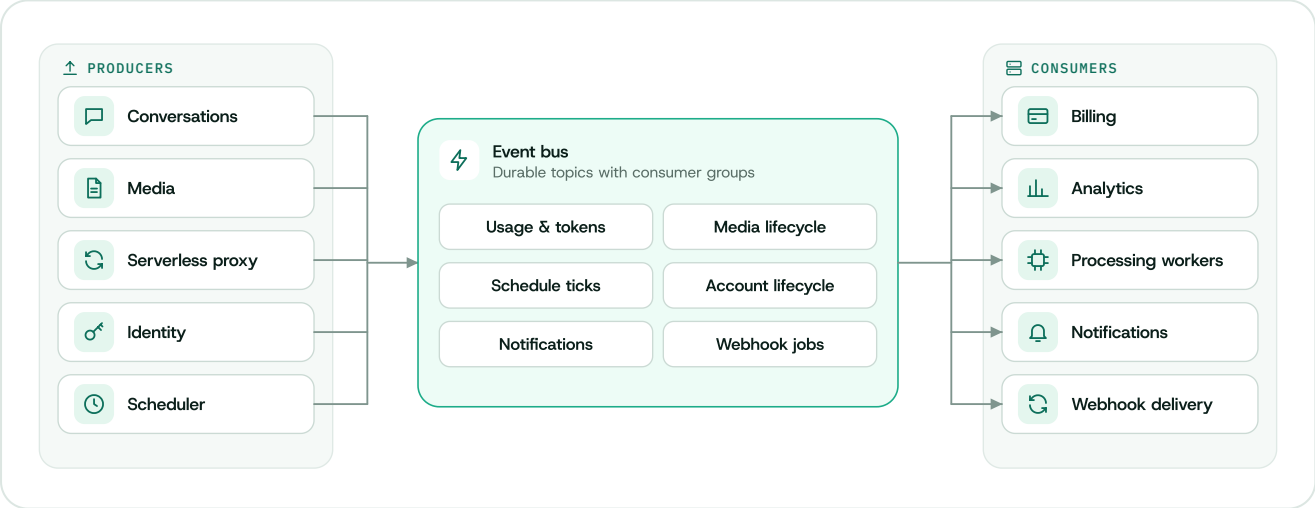



FIGURE 9 Event-driven integration: producers publish domain events; billing, analytics, workers and notifications consume them independently.

Externally, the platform offers a documented public API with scoped API keys, an SDK, webhooks for asynchronous results, an embeddable chat widget, Slack and email channels, and an MCP interface that exposes platform capabilities to agents.

09

# Data architecture

Each service owns its data. The platform uses a small number of store types, each chosen for one kind of workload.

| Store | Holds | Protection |
| --- | --- | --- |
| **Operational document database** | System of record per domain: organisations and users, conversations, agents, workflows, billing, configuration | Tenant-scoped records; snapshot backups |
| **Cache and messaging** | Caches, event bus, job queues, live channels, rate-limit counters, resumable stream buffers | Managed service with persistence |
| **Vector stores** | Embeddings for knowledge, media and agent discovery | Separate namespace per organisation |
| **Search index** | Agent marketplace and catalogue search | Derived data; rebuildable |
| **Object storage** | Uploaded media, generated files and reports, skill bundles | Signed, time-limited access; versioning, soft delete, point-in-time restore |
| **Secret vault** | Customer tool credentials and platform secrets | Managed key vault; access audited |

## Tenancy and lifecycle

- **Isolation:** the organisation identifier is part of every request, record, API key and vector namespace. Services enforce it in every read and write.
- **Lifecycle:** scheduled jobs remove expired sessions, batches and temporary data. Deleting an account triggers clean-up events in every domain that holds that organisation's data.
- **Portability:** storage sits behind an adapter that supports major cloud object stores and S3-compatible on-premises storage.

10

# Identity, access and tenancy

Users sign in with email and password or with single sign-on from major identity providers. Multi-factor authentication and bot protection are supported. The identity service issues short-lived access tokens and refresh tokens. Programmatic access uses API keys that belong to an organisation and can be scoped and revoked.

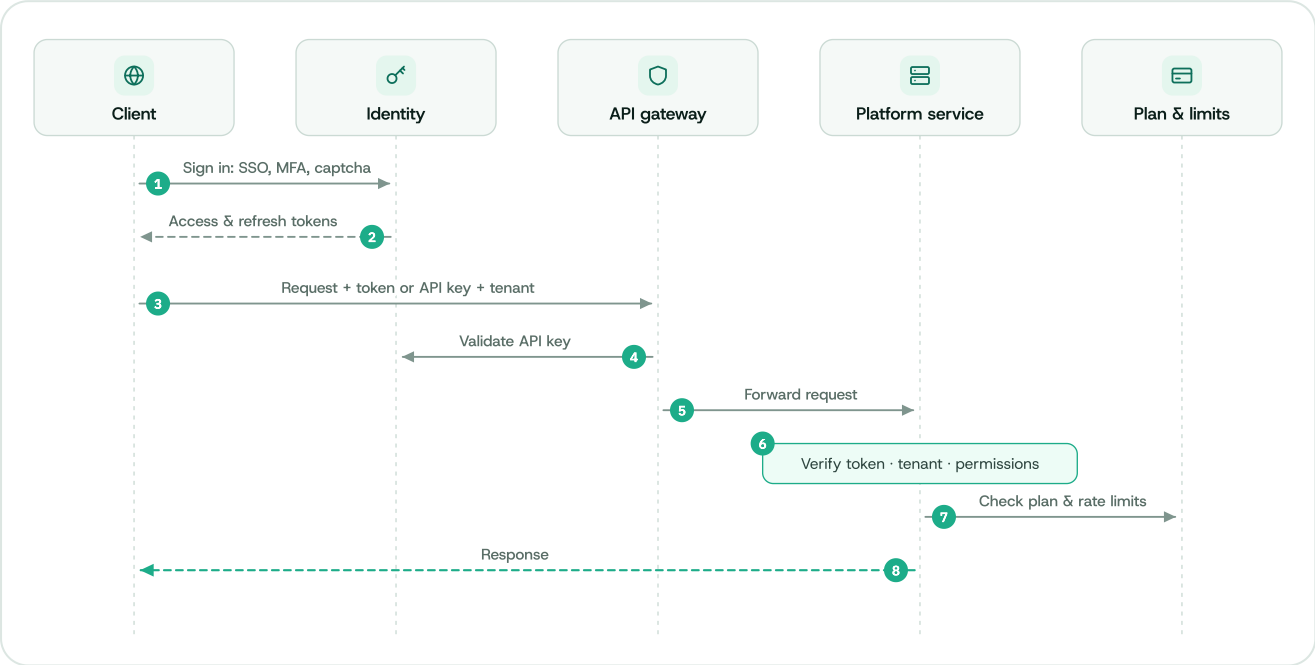



FIGURE 10 Identity and access: sign-in, gateway validation, service-level authorisation and plan checks.

- **Organisations** are the tenant boundary. A user can belong to several and switch between them.
- **Roles and permissions** control access to features and individual resources. Invitations, active-session review and remote sign-out are built in.
- **Two layers of enforcement.** The gateway validates API keys and administrative access before routing. Each service verifies the token, sets the tenant context and checks permissions.
- **Commercial controls.** Before any billable work, services confirm the organisation's plan, credit and rate limits. Unpaid balances lead to controlled suspension.

11

# Security architecture

Security is applied in layers, from the edge to the build pipeline, so that no single control is solely responsible for protecting customer data. The controls below describe the design. Detailed configuration is kept in internal documentation and shared under separate agreement.

### Edge

- Single API gateway
- TLS termination
- Cross-origin policy
- Per-client rate limiting
- Key & admin validation

### Identity & access

- SSO & MFA
- Bot protection
- Short-lived tokens
- Scoped API keys
- Role-based access
- Tenant context

### Application

- Plan & limit checks
- Outbound request (SSRF) guard
- Skill security scanning
- Marketplace moderation
- Content-security policy

### Data

- Per-tenant isolation
- Signed, expiring file access
- Credential vault
- Backups & point-in-time restore

### Platform

- Cluster SSO & RBAC
- Restricted pod security
- Workload identity
- Secrets mounted from vault
- Isolated agent sandboxes

### Supply chain

- Secret scanning
- Static analysis
- Dependency & image CVE gates
- Two-person bypass control
- Reproducible, tagged builds

### Detection & audit

- Runtime threat protection
- Vault access audit
- Central logs, metrics & traces
- Operational alerting

HANDLING OF SECRETS

Secrets are never stored in source control. Scanning blocks any change that contains one. Runtime secrets come from a managed key vault or the cluster's secret store, and build systems reach the container registry with workload identity instead of stored passwords.

12

# Infrastructure and deployment

The managed service runs on Microsoft Azure. Application services run on Azure Kubernetes Service. Each product line has its own production cluster, development workloads share a separate cluster, and a platform cluster hosts build and observability tooling. The web console is served from a global edge network.

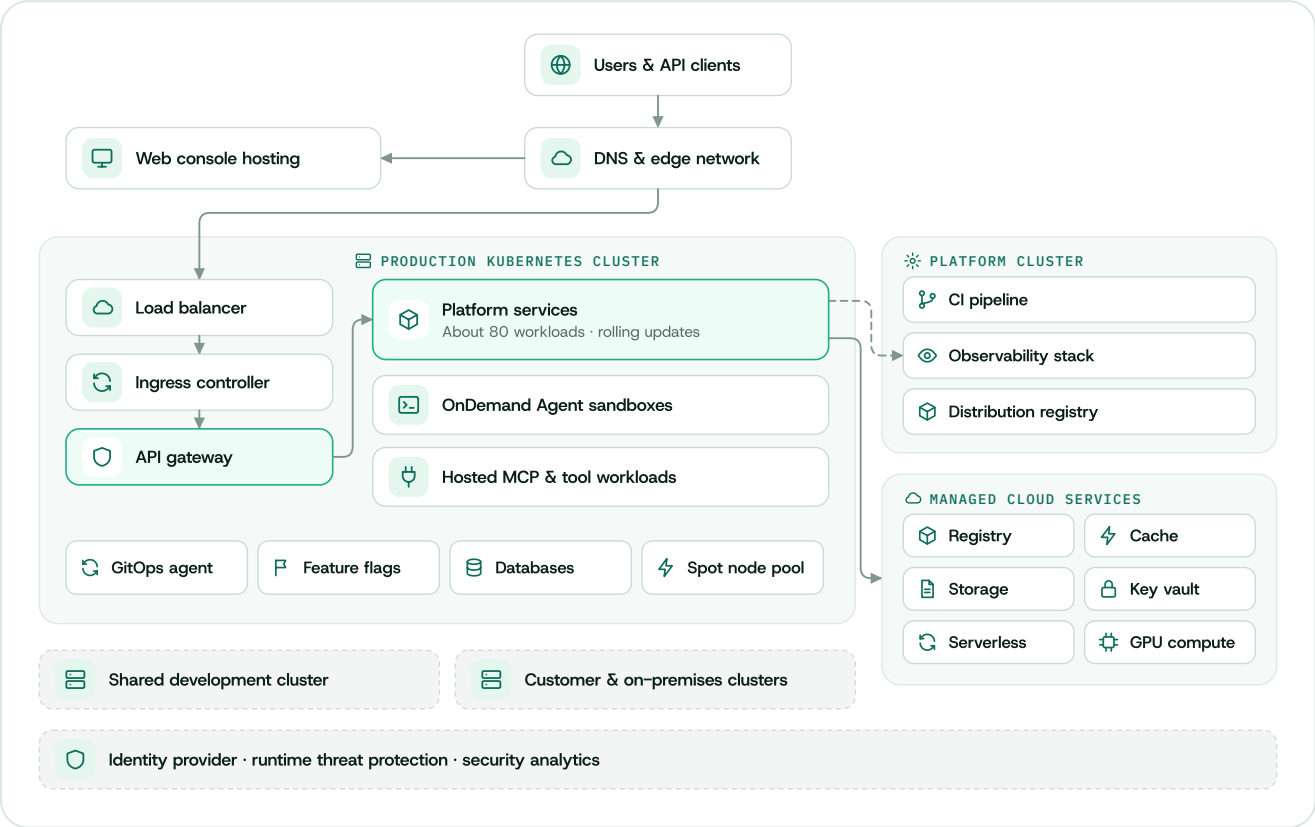



FIGURE 11 Deployment topology: edge path, production cluster, platform cluster and managed cloud services.

- **Ingress path:** a cloud load balancer passes traffic to a managed ingress controller, which forwards it to the API gateway, which routes it to services by path.
- **Workloads:** about 80 service deployments with fixed replica counts per environment. Non-critical services can use a spot-first node pool that falls back to on-demand capacity.
- **Dynamic workloads:** OnDemand Agent sandboxes, hosted MCP servers and browser tools are created on demand and removed when finished or idle.
- **Configuration:** services read configuration and secrets at start-up. Feature flags let behaviour change without a redeploy.

## Environments

| Environment | Purpose | How changes arrive |
|---|---|---|
| **Development** | Integration of all product lines on development builds | Automatically from the development branch |
| **Production** | Customer-facing service, one cluster per product line | Automatically on merge to the main branch, after all gates pass |
| **Platform** | Build system and observability stack | Managed by the platform team |
| **Customer / on-premises** | The product in a customer's own environment | Installed from a distribution registry, online or from an offline bundle |
| **Web console** | The browser application | Separate release path on the edge hosting provider |

## Deployment models

| Model | Description |
|---|---|
| **Managed cloud (SaaS)** | Operated by OnDemand on Azure; multi-tenant; continuous delivery. |
| **Customer cloud / on-premises** | The same services packaged as container images and Helm charts, installed in dependency order. Storage and serverless workloads use on-premises equivalents. |
| **Air-gapped** | A self-contained offline bundle of images and charts for sites without internet access. |
| **White-label** | Branding (logo, colours, available features) can be set per deployment. |

13

# Delivery and release management

Releases follow a pull-based GitOps model. The build system builds and checks each change, then records the exact image version in a Git repository that describes the desired state of each environment. An agent inside each cluster pulls that repository and applies it. Nothing pushes into application clusters, and every running version can be traced to a reviewed commit.

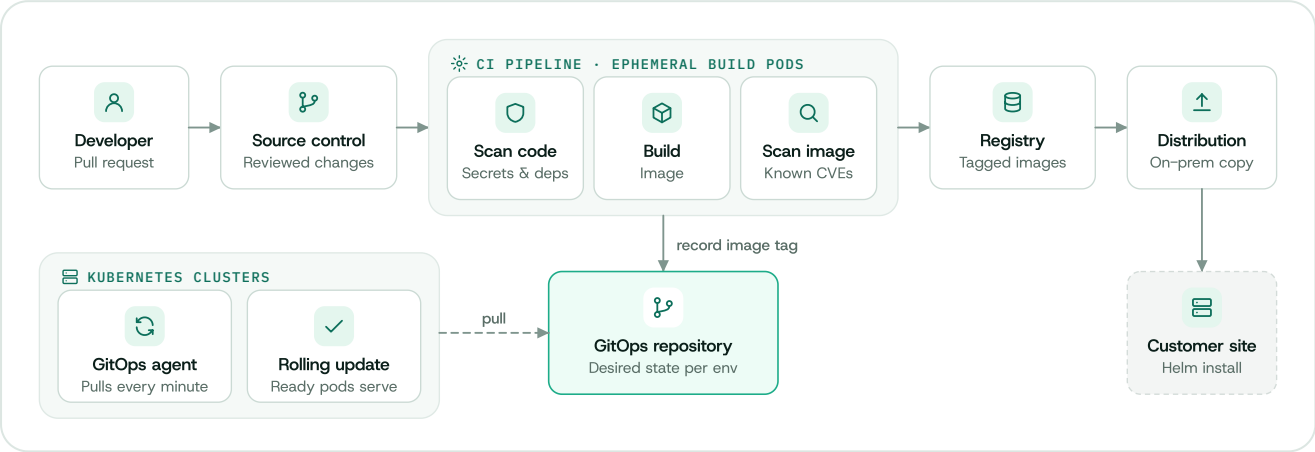



FIGURE 12 Delivery pipeline: gated builds, a single image registry, Git-recorded desired state and in-cluster reconciliation.

- **One shared pipeline** serves about 100 service repositories. Adding a new service takes almost no pipeline work.
- **Gates on every pull request:** secret scanning, static analysis and dependency vulnerability scanning. A failure blocks the merge. Skipping the dependency scan requires a written reason and approval from a second person, and both are recorded.
- **Gates on release:** the built image is scanned again, and fixable vulnerabilities block production releases.
- **Traceable versions:** images are tagged with their source commit. Rolling back means restoring a previously released, already-scanned image by reverting one Git commit.
- **Gateway configuration** follows its own automated path: validate, back up, apply, smoke test, and roll back automatically on failure.

## Target state: progressive delivery

The next stage keeps the current pipeline and adds progressive delivery. A new version is first tested on a preview address, then exposed to a growing share of traffic. It is promoted only if its success rate, latency and resource use stay within limits compared with the current version. Otherwise it is rolled back automatically, without a Git revert.

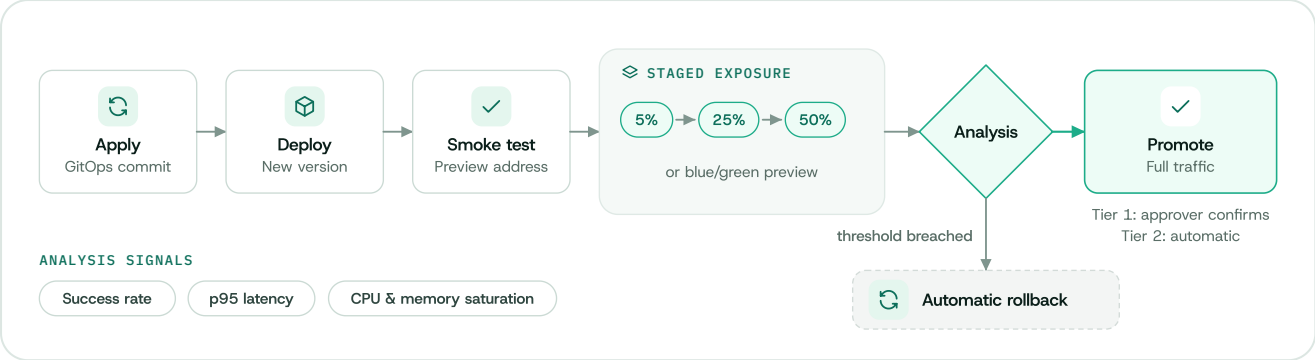



FIGURE 13 Target progressive delivery: staged exposure with automated analysis, tiered promotion and automatic rollback.

| Change type | Release strategy | Rollback |
|---|---|---|
| **Product behaviour** | Feature flag, enabled per group of users | Instant (flag off) |
| **User-facing service release** | Blue/green with a preview address | Seconds (switch back) |
| **Runtime, dependency or configuration change** | Canary at 5%, then 25%, then 50% of traffic | Seconds (traffic weight to zero) |
| **Architectural change** | Staged cutover with the old path kept available | About a minute |

Services are tiered. **Tier 1** services are on the customer request path or handle billing or licence data, so a named approver confirms each promotion. **Tier 2** services (background workers, schedulers, internal tools) are promoted automatically when analysis passes. Database changes follow an expand, migrate, contract pattern, so old and new versions can run side by side.

# Observability, resilience and continuity

- **Telemetry:** services emit OpenTelemetry traces, metrics and logs, with trace context carried across HTTP and RPC calls. A central stack stores and visualises them, alongside application performance monitoring.
- **Alerting:** operational events and build outcomes go to team channels. Runtime threat protection watches images and clusters. Access to the key vault is audited.
- **Health:** services expose readiness and liveness endpoints. The gateway exposes its own health probe for the load balancer.
- **Work resilience:** durable queues retry failed jobs, streams resume after disconnects, stale work is cleaned up automatically, and query cancellation reaches every worker.
- **Data protection:** snapshot backups for in-cluster databases, persistence for the managed cache, and versioning with point-in-time restore for object storage.

| Recovery scenario | Time to restore today | Mechanism |
|---|---|---|
| Faulty service release | ~1–2.5 min | Revert the release commit; the cluster restores the previous scanned image |
| Faulty release needing a rebuild | ~4–11 min | Revert the source change; the full pipeline runs again |
| Gateway configuration error | seconds – 1 min | Automatic rollback on failed smoke test |
| Web console release | minutes | Hosting provider's instant rollback |
| Target for service releases | seconds | Flag off, blue/green switch-back or canary weight to zero |

# Architecture roadmap

The roadmap builds on the current architecture rather than replacing it. Foundations come first, because progressive delivery depends on reliable health signals.

| Theme | Planned improvement | Outcome |
|---|---|---|
| Foundations | Readiness and liveness probes, disruption budgets and resource requests on every service; at least two replicas for customer-facing services | Safe rolling updates and node maintenance |
| Foundations | Deployment-outcome notifications, immediate apply on commit, deployment markers on dashboards | Faster feedback; easier incident correlation |
| Foundations | One-step automated rollback; security scans on merged code as well as pull requests | Consistent recovery in about a minute |
| Progressive delivery | Blue/green for user-facing services; canary for runtime and configuration changes; tiered promotion | Changes observed before full exposure; rollback in seconds |
| Supply chain | Software bill of materials and image signing with provenance, verified before rollout | Proof of origin; fast vulnerability impact analysis |
| Visibility | Delivery metrics: deployment frequency, lead time, change failure rate, time to restore | Objective release performance tracking |
| Developer experience | Preview environment per pull request; automated dependency and base-image updates | Earlier feedback; fewer emergency upgrades |

# Appendix

## A. Technology summary

| Area | Technologies (by category) |
|---|---|
| **Web client** | React and TypeScript single-page application |
| **Services** | Go microservices (HTTP and RPC); Python for query planning; Node.js for document processing and notifications |
| **AI** | Commercial model providers, real-time voice models, customer-hosted open models on GPUs, Model Context Protocol |
| **Data** | Document database, in-memory cache and streams, vector databases, search index, object storage, key vault, MQTT broker |
| **Platform** | Azure Kubernetes Service, managed ingress, API gateway, container registry, managed container apps, spot-capable node pools |
| **Delivery** | Git hosting, shared CI pipeline, secret/SAST/CVE scanners, container builds, Kustomize overlays, in-cluster GitOps agent, Helm for customer installs |
| **Observability** | OpenTelemetry, metrics, logs and traces stack, APM, runtime threat protection, security analytics |

## B. Glossary

| Term | Meaning |
|---|---|
| **Agent** | A tool the planner can use: an API, knowledge base, device, MCP server or built-in capability. Called a plugin in the API. |
| **Skill** | A written playbook, with optional helper files, that tells agents how to carry out a task. |
| **Endpoint** | A platform identifier for a model configuration. It hides which provider, or which customer deployment, serves the model. |
| **Reasoning mode** | A strategy the planner uses to break down and route a request. Organisations can define their own. |
| **Orchestration** | The staged execution of a query: plan, run steps, fulfil. |
| **OnDemand Agent** | The platform's autonomous agent. It runs each task in an isolated sandbox with code, files, a browser and skills, and can use the platform's own agents through MCP. |
| **Fulfilment** | The final step in which a model writes the answer from the step results. |

| Term | Meaning |
| --- | --- |
| MCP | Model Context Protocol, an open standard for exposing tools and data to AI agents. |
| BYOM / BYOI | Bring your own model / bring your own inference endpoint. |
| GitOps | Operating model in which Git holds the desired state and an in-cluster agent applies it. |
| Organisation | The tenant: the unit of isolation, billing and administration. |